

Kanal	Typ	Wertebereich	Belegung	alternativ Spieltisch 2
1	Note	21-108 (88 Töne, Piano) 36-91 (56 Töne, Orgel) 36-93 (58 Töne, Orgel) 36-96 (61 Töne, Orgel)	Manual I	Wenn möglich, wird ein zweiter Spieltisch über einen eigenen UDP-Port mit gleicher Kanalbelegung wie 1. Spieltisch angebunden. Diese Option wird genutzt, falls nur ein Interface möglich ist.
2	Note	dto.	Manual II	
3	Note	dto.	Manual III	
4	Note	dto.	Manual IV	
5	Note	dto.	Manual V	Manual I
6	Note	dto.	(Manual VI)	Manual II
7	Note	dto.	(Manual VII)	Manual III
8	Note	36-65 (67)	Pedal	
9	Note	36-65 (67)	nicht belegt	Pedal
10	<i>nicht belegt, da MIDI-Klangerzeuger hier oft Percussions-Sounds abspielen</i>			
11	Note	1-126	Register 1-126	
	Controller	0-127	7-18: Expr. Pedal 1-12	
			47-58: Replay Expr. 1-12	
			111: Spieltisch-Bedienelemente	
			112: Crescendo	
			0, 32: Kombination (Bank select)	
	program change		Kombination	
12	Note	1-126	Register 127-252	
	Controller	0-127	(nur für option Zweitspieltisch)	7-18: Expr. Pedal 1-12
				47-58: Replay Expr. 1-12
				111: Spieltisch-Bedienelemente
				112: Crescendo
13	Note	1-126	Register 253-378	
14	Note	1-126	Register 379-504	
15	Note	1-126	Register 505-630	
16	Note/Ctr.	diverse	nicht-standardisierte Daten	
	Sysex	in Diskussion	Anlagen- oder Hersteller-spezifische Steuerdaten	
egal	Controller	Controller 123	AllNotesOff Zurücksetzen aller aktiven Töne und Register (optional einstellbar)	

übliche UDP-Port Nutzung (z.B. Laukhuff Anlagen)

ipMIDI-Port	UDP-Port	Verwendung
1	21928	Spieltisch 1 -> SPS
2	21929	Spieltisch 2 -> SPS
3	21930	opt. Stimmsystem -> SPS / Spieltisch 3 -> SPS
4	21931	opt. externer MIDI-In -> SPS
5	21932	SPS -> Trakturausgänge oder ext. MIDI-out

Externe Steuerung von Setzeranlagen (z. B. Sinua)

Vom Setzer wird über ein nacheinander gesendetes Paar einer MIDI Bank-select und einer Program Change Message die gewünschte Kombination abgerufen

bank select	Dezimal-Setzer	Alpha-Nummerischer Setzer (A-H, 1-8)
MSB	100k, 10k, 1k	memory level 0...127
LSB	100, 10	A, B, C,
program change		
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10		10
...		...

Klassifikation von Midi-Implementationen im Orgelbau

MIDI Level	Features	Anwendungen
0	Die Anlage gibt die Daten der Tontraktur aus.	Das Spiel kann aufgezeichnet werden, um z.B. Noten daraus zu erzeugen oder es sich auf heimischen Soundgeneratoren anzuhören. Mit Einschränkungen kann man auch die Tontraktur extern ansteuern, indem man Traktur-kompatible Daten einspeist und die Orgel so reagiert, als würden die Töne live auf den Manualen gespielt
1	Die Anlage gibt die Daten von Ton- und Registertraktur aus.	s.o. mit Ergänzung der Registerdaten, die bei geeigneten externen Geräten die Steuerung von Klangerzeugern ermöglicht.
2 	Die Anlage gibt die Daten von Ton- und Registertraktur aus und kann diese Daten auch empfangen .	Damit steht eine bidirektionale Schnittstelle zur Verfügung, über die ein externer Recorder das Spiel vollständig aufzeichnen und wiedergeben kann. Außerdem sind damit alle Daten im ipMidi-Format im Netzwerk vorhanden, mit denen später ohne Eingriffe in die Originalsteuerung Erweiterungen angesteuert werden können (externe Spieltische, Auxiliarwerke etc.). Einschränkungen bestehen, wenn das Steuersystem interne Verarbeitungen mit hart verkabelten Schaltern steuert, z.B. Transpositeur, Sostenuto und ähnliches. Man kann aber im Wesentlichen alle Orgelfunktionen von außen ansteuern.
3 	Alle Traktur- und Steuersignale werden offen im ipMIDI-Format im Orgelnetzwerk kommuniziert.	Damit können einzelne Steuerungsteile jederzeit Hersteller-unabhängig ausgetauscht werden. Man könnte z.B. Windladenmodule durch Wago-Inseln ersetzen oder ein dezentral gesteuertes Fernwerk durch ein neues. Signale können von zukünftigen Steuerungsteilen gelesen und wenn nötig modifiziert und weitergeleitet werden. Beispielsweise kann eine externe CPU im Netz Daten lesen und modifizierte Daten über einen anderen UDP-Port oder MIDI-Kanal senden, auf die dann neue Anlagenteile reagieren.

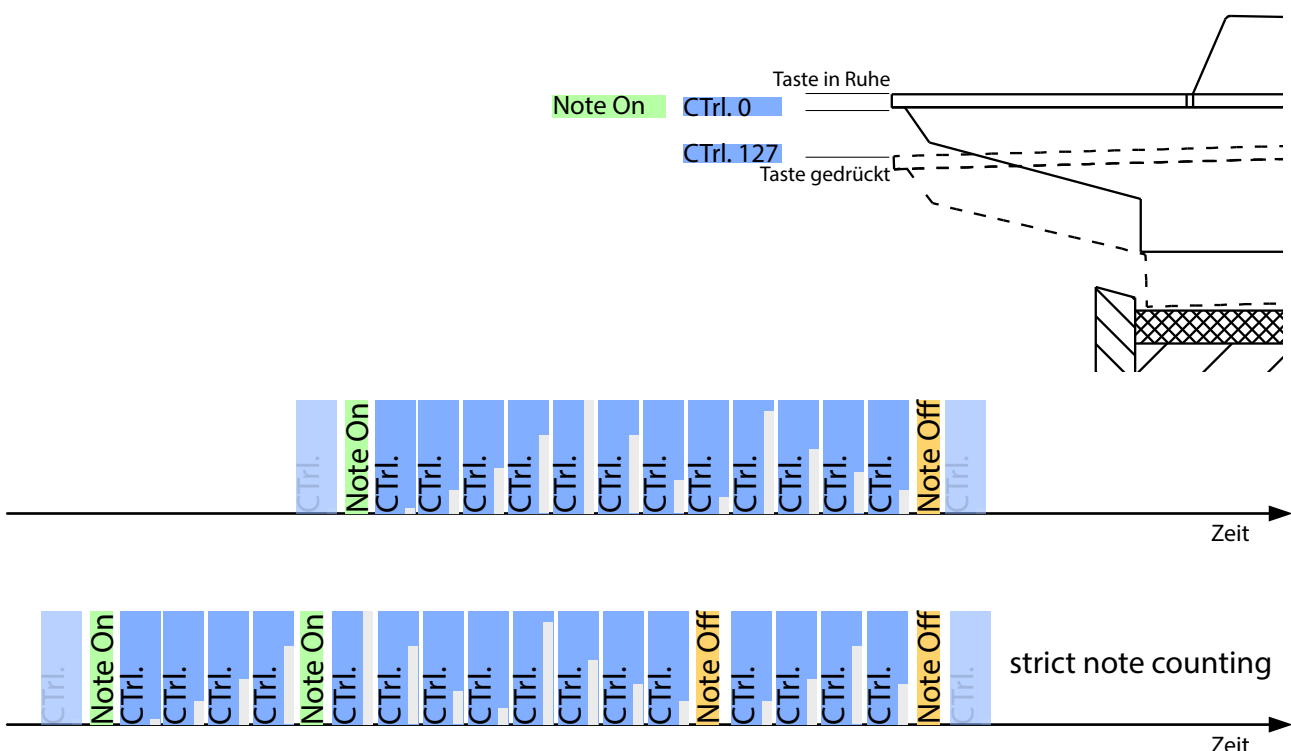
Variable Tontraktur mit MIDI Controllern

Tontrakturen, die eine graduelle Steuerung von Ventilen ermöglichen, tun dies über Controller Messages, die zusätzlich zu den Note-On- und Note-Off-Messages gesendet werden. Die Controller senden in nicht festgelegten Zeitabständen Änderungen der Tastenposition.

Dabei gilt:

- Auf einem MIDI-Kanal ist jeder Note ein Controller zugeordnet, der dieselbe Nummer hat wie die Note. So wird z.B. Note 36 (C) von Controller 36 gesteuert.
- Die Note-On-Message und die Note-Off-Message definieren den Beginn und das Ende des Tons.
- Werden mehrere Note-Ons in Folge gesendet, können Empfänger diese zählen und den Ton erst beenden, wenn die entsprechende Zahl von Note-Offs eingegangen ist. Dieses „strict note counting“ wird z.B. vom Espressivo System verwendet, um die Tonlochmaskierung auszulösen und die korrekte Funktion von MIDI-Koppeln zu gewährleisten.
- Controller-Daten, die vor dem ersten Note-On oder nach dem letzten Note-Off empfangen werden, haben keine Wirkung.
- Ein Note-Off beendet den Ton, auch wenn der Controller aktuell noch Werte > 0 haben sollte.
- Controller-Werte folgen der Tastenbewegung. D.h. Die Tastenposition unmittelbar an der Note-On-Position entspricht dem Controller-Wert 0, die maximal gedrückte Taste entspricht dem Controller-Wert 127.

Die Positionsänderung einer Taste sollte nicht häufiger als alle 10ms übertragen werden. Höhere Update-Raten sind mechanisch irrelevant und erzeugen unnötig hohen Datenverkehr.



ipMIDI Datenformat und Timing

ipMIDI überträgt das klassische 3-Byte-Format in UDP-Paketen via Ethernet, vorzugsweise mit 100MBit/s Datenrate.

Dabei muss ein Kompromiss zwischen minimaler Latenz und akzeptabler Datenmenge gefunden werden:

Eine zu hohe Paketrage kann Empfänger mit begrenzter Performance (Microcontroller) überfordern und zu Paketverlust führen.

Zyklus-basierte SPS-Systeme können oft nur ein Ethernet-Paket pro Zyklus verarbeiten, so dass eine hohe Paketrage paradoxerweise zu erhöhter Latenz führt.

Die Einhaltung eines zeitlichen Mindestabstands zwischen den Paketen vermeidet übermäßige Last im Netzwerkstack auf der Empfängerseite, und die Übermittlung mehrerer MIDI-Messages in einem UDP-Paket sorgt für kurze Latenz.

Daher sind folgende Randbedingungen einzuhalten:

- ipMIDI Sender halten einen **minimalen Zeitabstand von 1ms** zwischen Paketen ein.
- ipMIDI Sender können bis zu **85 MIDI-Messages in einem ipMIDI-Datenpaket** senden (255 Datenbytes)
- Eine MIDI-Message besteht immer aus 3 Byte, Kompression wie „running status“ wird nicht verwendet.
- ipMIDI-Empfänger müssen bis zu **1000 MIDI-Messages/s bei voller Funktion** verarbeiten können
- ipMIDI-Empfänger müssen bis zu **3000 MIDI-Message/s ohne Fehlfunktion** (Notenhänger) verarbeiten können. Dies kann z.B. durch „graceful failure“-Mechanismen gewährleistet werden, die an der Performance-Grenze Note-Off-Message priorisieren und Note-On-Messages verwerfen.
- ipMIDI-Empfänger müssen **10.000 MIDI Message/s im UDP-Netzwerk** ertragen ohne Fehlfunktion.

erweitertes ipMIDI-Format für große Adressräume

Die standardisierte Belegung von ipMIDI-Kanälen bietet ausreichend Adressraum für alle sinnvoll denkbaren Arten von Spieltischen und Orgelwerken. Die Beschränkung auf 9 Tontrakturkanäle stößt jedoch an Grenzen, wenn eine größere Anzahl von Einzeltonladen die individuelle Ansteuerung einzelner Ausgänge erlauben soll.

Es wäre möglich, mehrere UDP-Ports zu verwenden, um den Adressraum zu vergrößern. Dies behindert aber verteilte Architekturen, in denen alle Informationen allen Empfängerstationen zur Verfügung stehen sollen.

Daher wird in diesen Fällen das 3-Byte-MIDI-Format um ein 4. Byte erweitert, den „MIDI-Port“.

ipMIDlx-Format (IPMX) umfasst 64 Ports. Bleibt auf Port 0 reserviert für die standardisierte Kodierung von Ton- und Registertraktur, können auf Ports 1-63 unter Nutzung aller MIDI-Kanäle und Tonumfänge weitere 129.024 Einzelausgänge adressiert werden.

MIDI-Status	2. Byte	3. Byte	MIDI Port
Status Note an/aus Kanal 0...15	Noten Nummer 0..127	Velocity 0..127	Port 0...63

Der Wertebereich 64-127 des 4. Bytes im IPMX-Format bleibt derzeit frei für weitere Funktionen.

Hierarchisch gesehen ist der Port die höchste Instanz, so dass jeder Port weitere 16 MIDI-Kanäle öffnet. Bei der Übertragung wird die Portnummer als letztes Byte angehängt. So können bestehende MIDI-Parsing-Algorithmen leicht zur Dekodierung des Port-Bytes erweitert werden. Da Statusbytes die einzigen Bytes sind, die das MSB verwenden, werden sie zur Erkennung des Beginns einer MIDI-Nachricht verwendet.

ipMIDI- und ipMIDlx-Daten

können alternativ auch im standardisierten OSC-Format kodiert werden:

/IPM, status, byte_2, byte_3

/IPMX, status, byte_2, byte_3, port

Channel	Type	Values	Assignment	alternative
1	Note	21-108 (88 Notes, Piano) 36-91 (56 Notes, Organ) 36-93 (58 Notes, Organ) 36-96 (61 Notes, Organ)	Manual I	
2	Note	dto.	Manual II	
3	Note	dto.	Manual III	
4	Note	dto.	Manual IV	
5	Note	dto.	Manual V	Console 2, Manual I
6	Note	dto.	(Manual VI)	Console 2, Manual II
7	Note	dto.	(Manual VII)	Console 2, Manual III
8	Note	36-65 (67)	Pedal	
9	Note	36-65 (67)		Console 2, Pedal
10	<i>don't use (multitimbral devices play percussion sounds by default)</i>			
11	Note	1-126	Stops 1-126	
	Controller	0-127	Controller 7: Swell 1	
			Controller 8: Swell 2	
			Controller 9: Swell 3	
12	Note	1-126	Stops 127-252	Console 2
	Controller	0-127	Controller 7: Swell 4	Controller 7: Swell 1
			Controller 8: Swell 5	Controller 8: Swell 2
			Controller 9: Swell 6	Controller 9: Swell 3
13	Note	1-126	Stops 253-378	
14	Note	1-126	Stops 379-504	
15	Note	1-126	Stops 505-630	
16	Note	diverse	Action functions, e.g. sequencer buttons	
	Controller	diverse		
	Sysex	in discussion	System or vendor-specific data, e.g. to recall specific registrations in capture systems	
n/a	Controller	Controller 123	AllNotesOff	
			Reset all active notes and stops (selectable as an option)	

common UDP-Port assignment

ipMIDI-Port	UDP-Port	use
1	21928	console 1 -> PLC
2	21929	console 2 -> PLC
3	21930	opt. tuning system -> PLC / console 3 -> PLC
4	21931	opt. external MIDI-In -> PLC
5	21932	PLC -> action subsystem / distributed controllers or external MIDI-out

External control of capture systems (e.g. Sinua)

Registrations are recalled from the capture system by sending a pair of MITI bank-select and program change messages.

bank select	decimal stop action	alphanumeric action (A-H, 1-8)
MSB	100k, 10k, 1k	memory level 0...127
LSB	100, 10	A, B, C,
program change		
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10		10
...		...

Classification of MIDI-Implementations in Pipe Organ Systems

MIDI Level	Features	uses
0	The system transmits key action data.	Organ music can be recorded on external devices (e.g. converted to sheet music). External sound generators can be controlled. Some systems can be played by external signals (if the same data is used to feed the control system).
1	The system transmits key action and stop action data.	Dito with added stops data. Can be used to some extend to control external devices. Can be used to control other organs after data conversion.
2 	The system transmits data for Key action and Stop action and also receives this data.	Bi-directional interface. Can be used to record and playback performances. Can be used to control vendor-independent extensions to a system. Can be used to control essential function of the organ. Some restrictions on hard-wired functions.
3 	All action data (notes, stops, swells ...) is communicated between system components in an open MIDI-compatible format.	Functional units of the organ installation can be replaced independent of manufacturer. The organ control system can be extended without replacing existing hardware. Data can be read and modified by external devices to implement new capabilities.

Variable key action using MIDI Controllers

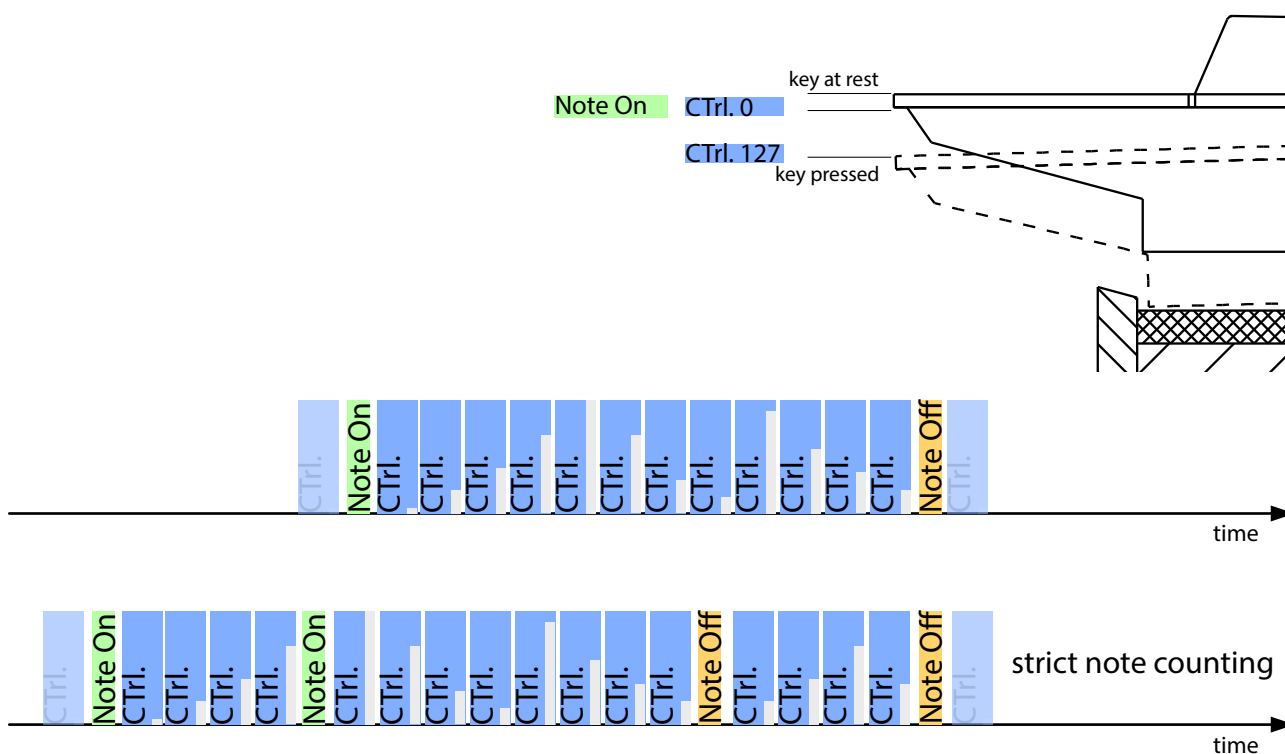
Key actions which allow for the gradual control of valves, do this via controller messages that are sent in addition to the note on and note off messages.

The controllers send changes to the key position at non-fixed time intervals.

The following applies:

- On a MIDI channel, each note is assigned a controller that has the same number as the note. E.g. note 36 (C) is controlled by controller 36.
- The note-on message and the note-off message define the beginning and the end of the tone.
- If several note-ons are sent in succession, receivers can count these and only stop the tone when the corresponding number of note-offs has been received. This “strict note counting” is used, for example, by the Espressivo system to trigger tone hole masking and to ensure that MIDI coupling functions correctly.
- Controller data received before the first note-on or after the last note-off has no effect.
- A note-off ends the tone, even if the controller should currently still have values > 0 .
- Controller values follow the key movement. This means that the key position immediately at the note-on position corresponds to the controller value 0, and the fully depressed key corresponds to the controller value 127.

The position change of a key should not be transmitted more frequently than every 10ms. Higher update rates are mechanically irrelevant and generate unnecessarily high data traffic.



ipMIDI Data format and Timing

ipMIDI transmits the classic 3-byte format in UDP packets via Ethernet, preferably at 100 Mbit/s data rate.

A compromise must be found between minimal latency and acceptable data volume:

A packet rate that is too high can overwhelm receivers with limited performance (microcontrollers) and lead to packet loss.

Cycle-based PLC systems can often only process one Ethernet packet per cycle, so that a high packet rate can paradoxically lead to increased latency.

Maintaining a minimum time interval between packets avoids excessive load on the network stack on the receiving side, and transmitting multiple MIDI messages in a single UDP packet ensures low latency.

Therefore, the following conditions must be met:

- ipMIDI transmitters maintain a minimum time interval of 1 ms between packets.
- ipMIDI transmitters can send up to 85 MIDI messages in an ipMIDI data packet (255 data bytes).
- A MIDI message always consists of 3 bytes; compression such as “running status” is not used.
- ipMIDI receivers must be able to process up to 1000 MIDI messages/s with full functionality
- ipMIDI receivers must be able to process up to 3000 MIDI messages/s without malfunction (stuck notes). This can be ensured, e.g., by “graceful failure” mechanisms that prioritize note-off messages and discard note-on messages at the performance limit.
- ipMIDI receivers must be able to handle 10,000 MIDI messages/s in the UDP network without malfunctioning.

extended ipMIDI format for large address spaces

The standardized assignment of ipMIDI channels provides sufficient address space for all conceivable types of consoles and organs. However, the limitation to 9 key action channels reaches its limits when a larger number of individual single-note chests are to allow individual control of individual outputs.

It would be possible to use multiple UDP ports to increase the address space. However, this hinders distributed architectures in which all information is to be available to all recipient stations.

Therefore, in these cases, the 3-byte MIDI format is extended by a 4th byte, the "MIDI port".

The ipMIDIx format (IPMX) includes 64 ports. If port 0 remains reserved for the standardized coding of note and stop actions, ports 1-63 can be used to address a further 129,024 individual outputs, using all MIDI channels and note ranges.

MIDI-Status	2. Byte	3. Byte	MIDI Port
Status Note on/off channel 0...15	note number 0..127	Velocity 0..127	Port 0...63

The range 64-127 of the 4th byte in the IPMX format currently remains free for further functions.

Hierarchically, the port is the highest instance, so that each port opens a further 16 MIDI channels. During transmission, the port number is appended as the last byte. This way, existing MIDI parsing algorithms can easily be extended to decode the port byte. Since status bytes are the only bytes that use the MSB, they are used to detect the beginning of a MIDI message.

ipMIDI and ipMIDIx data

can alternatively be encoded in the standardized OSC format:

/IPM, status, byte_2, byte_3

/IPMX, status, byte_2, byte_3, port